

Universidad Sergio Arboleda

Sistemas Operativos



Santiago Rodriguez

<https://github.com/SantiagoRodriguez114/AlgoritmoBanquero>

Bogotá, 2026

Introducción

El algoritmo del banquero, propuesto por Edsger Dijkstra, es una técnica de asignación de recursos utilizada en sistemas operativos para prevenir interbloqueos (deadlocks). Su principio fundamental consiste en permitir la asignación de recursos únicamente si el sistema puede garantizar que continuará en un estado seguro, es decir, en una condición donde todos los procesos puedan finalizar su ejecución sin quedar bloqueados.

A diferencia de otros enfoques que toman decisiones basadas únicamente en la disponibilidad inmediata de recursos, el algoritmo del banquero incorpora una verificación adicional que simula el comportamiento futuro del sistema. Esto le permite anticipar posibles conflictos y evitar situaciones en las que los procesos compiten por recursos de forma indefinida.

En el presente trabajo, este algoritmo se aplica en el contexto de una simulación de tráfico vehicular en una intersección de cuatro direcciones. Cada flujo de vehículos se modela como un proceso que requiere ciertos recursos para poder atravesar el cruce, tales como espacio en el centro de la intersección y capacidad en los ejes de circulación.

El objetivo de la simulación es demostrar cómo la lógica del algoritmo del banquero puede utilizarse para regular el acceso a un sistema compartido, garantizando su estabilidad y evitando estados inseguros.

Planteamiento del problema

La simulación se basa en una intersección donde convergen cuatro flujos de tráfico: dos horizontales y dos verticales. Cada uno de estos flujos se modela como un proceso independiente que requiere ciertos recursos para poder cruzar la intersección.

El sistema debe garantizar que el número de vehículos no exceda la capacidad del cruce, tanto en el eje horizontal como en el vertical, además de controlar el uso del espacio central, que representa la zona crítica donde pueden ocurrir conflictos.

En este contexto, surge la necesidad de implementar un mecanismo que no solo regula el paso de vehículos, sino que también evite estados problemáticos como la saturación o el bloqueo del sistema.

Modelo del sistema

Para representar la intersección, se definieron tres tipos de recursos: el espacio del centro del cruce, la capacidad del flujo horizontal y la capacidad del flujo vertical. Cada uno de estos recursos tiene un límite máximo que no puede ser sobrepasado durante la simulación.

A su vez, cada dirección de tráfico tiene una demanda específica de recursos, lo que determina si puede o no ingresar al sistema en un momento determinado. Estas demandas representan la cantidad de espacio necesario para que un grupo de vehículos atraviese completamente la intersección.

En este punto se puede observar cómo el problema del tráfico se traduce en un problema clásico de asignación de recursos.

```

TOTAL = [20, 50, 30]
|
PROCESOS = ["P1", "P2", "P3", "P4"]

# Demanda máxima
MAX = [
    [10, 25, 0], # P1
    [10, 25, 0], # P2
    [10, 0, 15], # P3
    [10, 0, 15], # P4
]

# Duración en ciclos
DUR = [3, 3, 2, 2]

alloc = [[0]*3 for _ in range(4)]
need = [row[:] for row in MAX]
rest = [None]*4
done = [False]*4

```

Aplicación del algoritmo del banquero

El algoritmo del banquero se integra en la simulación como el mecanismo encargado de decidir si un proceso puede acceder a los recursos del sistema. Cada vez que una dirección de tráfico intenta avanzar, el sistema realiza una verificación antes de permitir su entrada.

Primero, se evalúa si existen recursos suficientes disponibles. Sin embargo, este criterio por sí solo no es suficiente. Por esta razón, el sistema realiza una asignación temporal de los recursos y simula el comportamiento futuro de los procesos mediante una función que verifica si todos podrían terminar su ejecución.

Si esta simulación demuestra que el sistema puede continuar sin bloqueos, la asignación se mantiene. En caso contrario, se revierte la operación, evitando así entrar en un estado inseguro.

Este enfoque es clave, ya que introduce una visión preventiva: el sistema no toma decisiones únicamente con base en el estado actual, sino considerando las posibles consecuencias futuras.

```
def es_seguro():
    work = available()
    finish = [False]*4

    while True:
        progreso = False
        for i in range(4):
            if not finish[i] and all(need[i][j] <= work[j] for j in range(3)):
                for j in range(3):
                    work[j] += alloc[i][j]
                finish[i] = True
                progreso = True
        if not progreso:
            break

    return all(finish)
```

La función `es_seguro()` es el núcleo del algoritmo del banquero, ya que se encarga de determinar si el sistema se encuentra en un estado seguro.

Para ello, la función simula si todos los procesos pueden terminar con los recursos actualmente disponibles.

Primero, se calcula `work`, que representa una copia de los recursos disponibles en ese momento. También se crea la lista `finish`, que indica qué procesos han podido finalizar durante la simulación.

Luego, el algoritmo entra en un ciclo donde busca procesos que puedan completarse con los recursos actuales. Un proceso puede avanzar si su necesidad (need) es menor o igual a los recursos disponibles (work). Si esto se cumple, se asume que el proceso termina y libera sus recursos, los cuales se suman nuevamente a work.

Este procedimiento se repite hasta que no se pueda avanzar más. Es decir, cuando ya no hay procesos que puedan finalizar con los recursos disponibles.

Finalmente, si todos los procesos lograron terminar (finish en verdadero para todos), el sistema está en un estado seguro y la función retorna True. En caso contrario, retorna False, indicando que la asignación llevaría a un estado inseguro.

Control de semáforos

El sistema organiza el flujo vehicular en dos fases: una fase horizontal y una fase vertical. Durante cada fase, solo los procesos correspondientes pueden intentar acceder a los recursos, lo que simula el comportamiento básico de un sistema de semáforos.

No obstante, existe una diferencia importante respecto a un semáforo tradicional. En este modelo, el hecho de que una dirección tenga luz verde no implica que necesariamente pueda avanzar. La decisión final depende de la evaluación del algoritmo del banquero.

Esto significa que el sistema prioriza la estabilidad global sobre la continuidad del flujo inmediato, lo cual puede generar esperas incluso cuando aparentemente hay disponibilidad.

```
— Minuto 5 (HORIZONTAL) —————
P1:AMARILLO P2:AMARILLO P3:ROJO P4:ROJO
Disponible: [20, 50, 30]
P1 ENTRA - alloc=[10, 25, 0] (3 ciclos)
P2 ENTRA - alloc=[10, 25, 0] (3 ciclos)

— Minuto 10 (VERTICAL) —————
P1:ROJO P2:ROJO P3:AMARILLO P4:AMARILLO
Disponible: [0, 0, 30]
P3 ESPERA - sin recursos [0, 0, 30]
P4 ESPERA - sin recursos [0, 0, 30]

— Minuto 15 (HORIZONTAL) —————
P1:VERDE P2:VERDE P3:ROJO P4:ROJO
Disponible: [0, 0, 30]

— Minuto 20 (VERTICAL) —————
P1:ROJO P2:ROJO P3:AMARILLO P4:AMARILLO
Disponible: [0, 0, 30]
P1 COMPLETO - libera recursos
P2 COMPLETO - libera recursos
P4 ENTRA - alloc=[10, 0, 15] (2 ciclos)
P3 ENTRA - alloc=[10, 0, 15] (2 ciclos)

— Minuto 25 (HORIZONTAL) —————
P1:OFF P2:OFF P3:ROJO P4:ROJO
Disponible: [0, 50, 0]

— Minuto 30 (VERTICAL) —————
P1:OFF P2:OFF P3:VERDE P4:VERDE
Disponible: [0, 50, 0]
P3 COMPLETO - libera recursos
P4 COMPLETO - libera recursos

Completado en 30 minutos.
```

Ejecución de la simulación

La simulación se desarrolla en ciclos de tiempo discretos, donde cada iteración representa un intervalo de cinco minutos. En cada ciclo se actualiza el estado de los procesos activos, se liberan recursos cuando un proceso termina y se evalúan nuevas solicitudes de entrada.

El sistema imprime constantemente información relevante, como los recursos disponibles, el estado de los procesos y las decisiones tomadas en cada momento. Esto permite observar cómo evoluciona el sistema y cómo el algoritmo influye en el comportamiento del tráfico.

A lo largo de la ejecución, se evidencian situaciones en las que ciertos procesos deben esperar, incluso cuando hay recursos disponibles, debido a que permitir su entrada generaría un estado inseguro.

Conclusiones

El desarrollo de esta simulación permite evidenciar cómo un algoritmo clásico de sistemas operativos puede aplicarse a un problema cotidiano como la gestión del tráfico.

El algoritmo del banquero demuestra ser una herramienta efectiva para evitar estados inseguros, garantizando que todos los procesos puedan completar su ejecución sin generar bloqueos. Aunque el modelo utilizado es una simplificación, logra capturar la esencia del problema y muestra claramente la importancia de la planificación en la asignación de recursos.

Más allá del contexto vehicular, este ejercicio refuerza la comprensión de conceptos fundamentales como concurrencia, asignación segura y prevención de interbloqueos.

Fuentes

- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating system concepts (10th ed.). Wiley.
- Tanenbaum, A. S., & Bos, H. (2015). Modern operating systems (4th ed.). Pearson.
- GeeksforGeeks. (2024). Banker's algorithm in operating system.
<https://www.geeksforgeeks.org/bankers-algorithm-in-operating-system/>